CAPTURE THE FLAG

WRITEUP

Mobile Challenge

Locked... or Not? | DeadFace CTF 2026

Author	Md Solayman Hossain Shawon
Alias	CYB3R_B0T
Category	Mobile
Difficulty	Easy
Date Solved	19-10-2026
Status	Solved

1 Executive Summary

The **Locked... or Not?** challenge involved analyzing an Android application to identify a hidden flag. The investigation began by inspecting the provided **df-securenotes.apk.zip** archive and listing its contents using the **unzip -l** command. After identifying the APK file, I opened it using **JADX-GUI** and examined the decompiled source code. During the analysis of the **Main Activity**, I discovered a **Base64-encoded** string containing the flag. Decoding the string revealed the hidden flag. This challenge demonstrates how static analysis can uncover sensitive information embedded in an Android application's source code and highlights the importance of protecting secrets from direct exposure.

2 Challenge Information

Challenge Name	Locked... or Not?
Category	Mobile
Points	150
Difficulty	Easy
Files Provided	df-securenotes.apk.zip
Target	Find the hostname of investigated windows workstation as the flag
Tools	unzip,jadx-gui,base64
Author	Md Solayman Hossain Shawon

Challenge Description

The **Locked... or Not?** challenge involves analyzing an Android application named **df-securenotes.apk** to determine whether sensitive information is adequately protected. The challenge begins with a ZIP archive named **df-securenotes.apk.zip**, which contains the APK file. By extracting and examining the application, it is possible to inspect its internal structure and decompiled source code to identify potentially exposed information

Objective

The primary objective is to analyze the provided Android application using static analysis techniques, identify the **Base64-encoded** string embedded in the **Main Activity,** and decode it to retrieve the challenge flag. The challenge also aims to demonstrate the limitations of **Base64** encoding as a security mechanism and highlight the risks associated with embedding sensitive information directly within an application's source code.

3 Methodology / Approach

Summarize the strategy chosen and why, before walking through execution. This is the “how I thought about it” section.

Tools Used

Tool	Purpose
unzip	Used to inspect the APK archive contents and identify relevant files for further analysis.
jadx-gui	Used to decompile the Android APK and inspect the application's Java-like source code including the Main Activity, to identify the Base64-encoded flag.

Hypothesis

The Android application may contain sensitive information or the challenge flag within its decompiled source code. Since APK files can be inspected through reverse engineering techniques, examining the application's internal structure and Main Activity may reveal hardcoded or encoded data. It was hypothesized that the flag could be stored in Base64-encoded form and recovered through static analysis of the application.

4 Step-by-Step Walkthrough

Walkthrough the solve path in the order it happened with enough commands and screenshots that another player could reproduce it exactly.

Step 1 — Inspect the APK Archive

After the Unzip file, The analysis began by examining the provided df-securenotes.apk file to understand its internal structure. I used the following command to list the contents of the APK archive:

```
unzip -l df-securenotes.apk
```

The output revealed several important components including AndroidManifest.xml, application layout resources and the compiled DEX files (classes.dex and classes2.dex). These components indicated that the file contained the resources and bytecode required for further Android application analysis.

Step 2 — Analyze the Application Using JADX-GUI

Next, I opened the APK in JADX-GUI to decompile the application's bytecode and inspect its source code.

```
jadx-gui df-securenotes.apk
```

After the application loaded, I navigated to the **Main-Activity** and examined the decompiled code to identify potentially sensitive or encoded information.

Step 3 —Identify the Base64-Encoded Flag

While inspecting the **MainActivity**, I discovered a string encoded in **Base64** format. The presence of this encoded string suggested that the flag could be recovered by decoding it rather than by bypassing the application's interface or password protection.

Step 4 —Decode the Flag

I used the **Base64** decoder available in Kali Linux to decode the identified string.

```
echo 'ZGVhZGZhY2V7ZXotbW9iaWxlLXRvLXN0YXJ0fQ' | base64 -d
```

The decoded output revealed the challenge flag.

Step 5 —Verify the Result

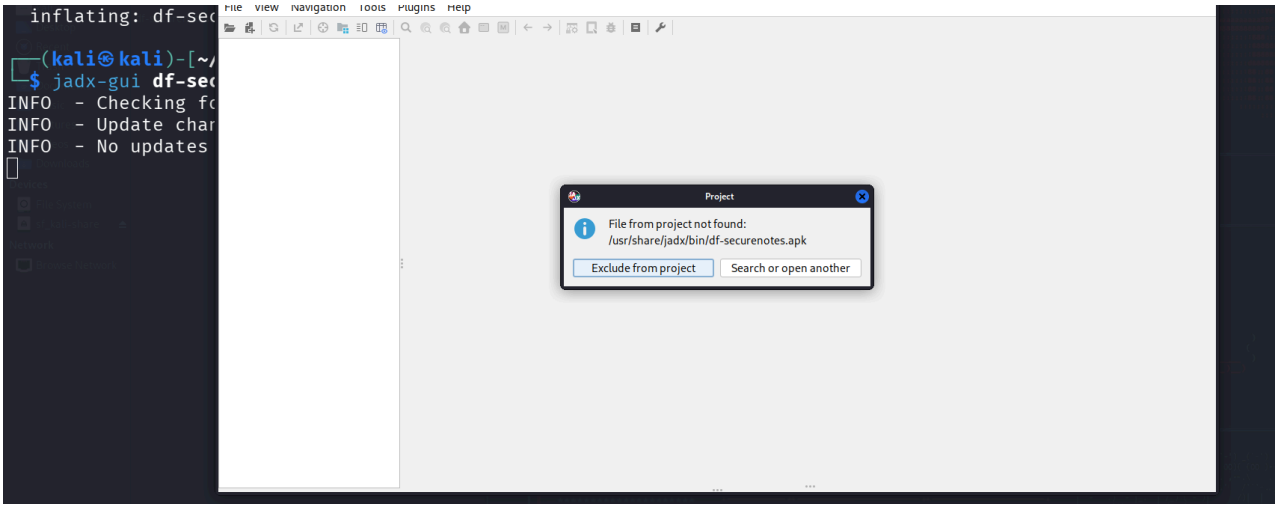
Finally, I checked the decoded output to confirm that it matched the expected flag format for the challenge. This completed the analysis and demonstrated that the flag could be recovered through static analysis of the APK.

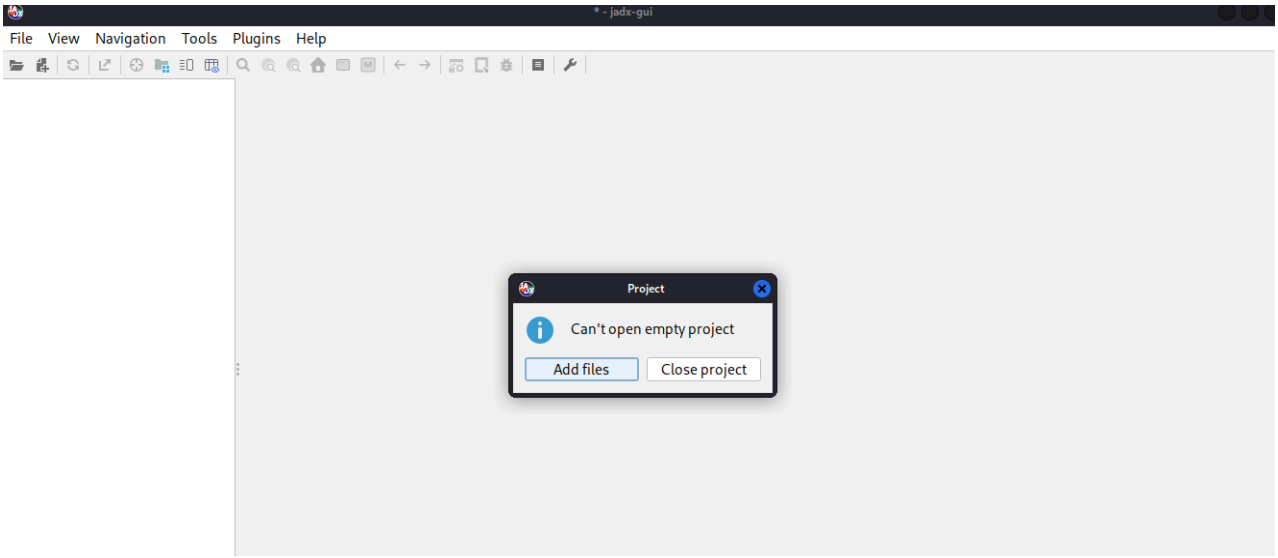
5 Proof of Concept

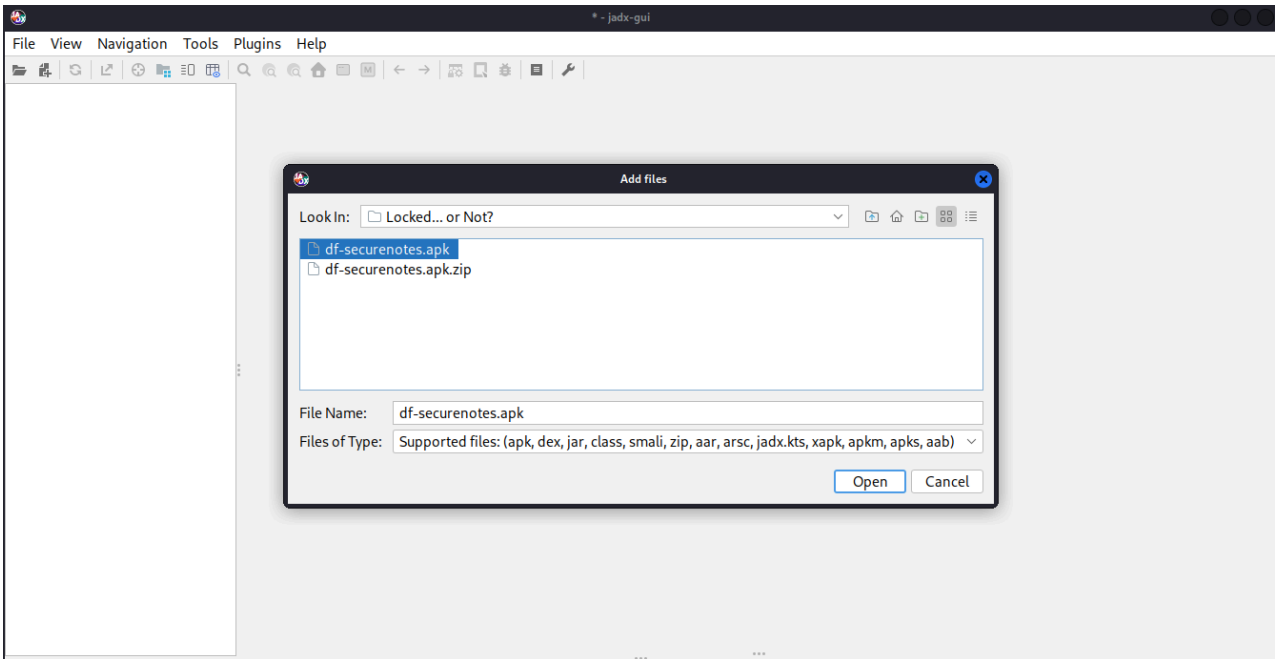
Step 1 — Inspect the APK Archive

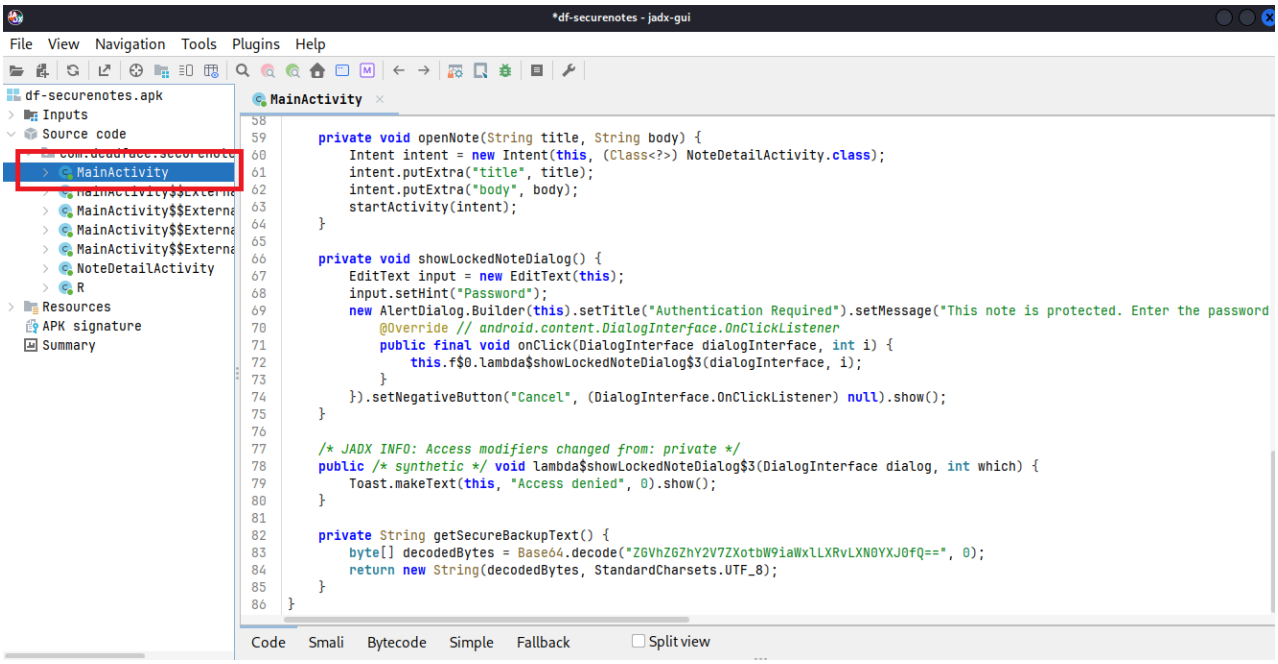


Step 2 — Analyze the Application Using JADX-GUI

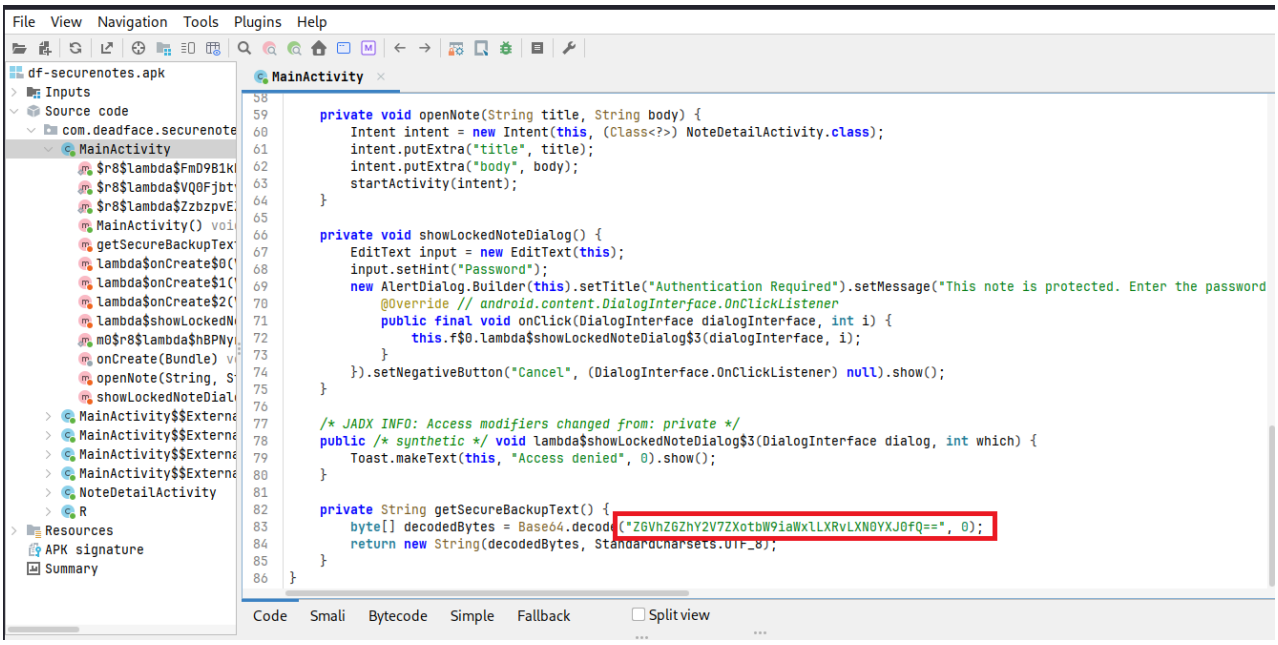




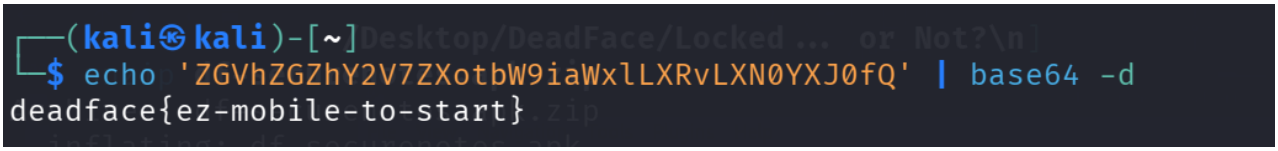

* - jadx-gui
File View Navigation Tools Plugins Help
Add files
Look In: Locked... or Not?
df-securenotes.apk
df-securenotes.apk.zip
File Name: df-securenotes.apk
Files of Type: Supported files: (apk, dex, jar, class, smali, zip, aar, arsc, jadx.kts, xapk, apkm, apks, aab)
Open
Cancel


*df-securenotes - jadx-gui
File View Navigation Tools Plugins Help
df-securenotes.apk
Inputs
Source code
MainActivity
MainActivity$$Externa
MainActivity$$Externa
MainActivity$$Externa
NoteDetailActivity
R
Resources
APK signature
Summary
MainActivity
private void openNote(String title, String body) {
Intent intent = new Intent(this, (Class<?>) NoteDetailActivity.class);
intent.putExtra("title", title);
intent.putExtra("body", body);
startActivity(intent);
}
private void showLockedNoteDialog() {
EditText input = new EditText(this);
input.setHint("Password");
new AlertDialog.Builder(this).setTitle("Authentication Required").setMessage("This note is protected. Enter the password
@Override // android.content.DialogInterface.OnClickListener
public final void onClick(DialogInterface dialogInterface, int i) {
this.f$0.lambda$showLockedNoteDialog$3(dialogInterface, i);
}
}).setNegativeButton("Cancel", (DialogInterface.OnClickListener) null).show();
}
/* JADX INFO: Access modifiers changed from: private */
public /* synthetic */ void lambda$showLockedNoteDialog$3(DialogInterface dialog, int which) {
Toast.makeText(this, "Access denied", 0).show();
}
private String getSecureBackupText() {
byte[] decodedBytes = Base64.decode("ZGVhZGZhY2V7ZXotbW9iaWxlLXRvLXN0YXJ0fQ==", 0);
return new String(decodedBytes, StandardCharsets.UTF_8);
}
}
Code Smali Bytecode Simple Fallback
Split view

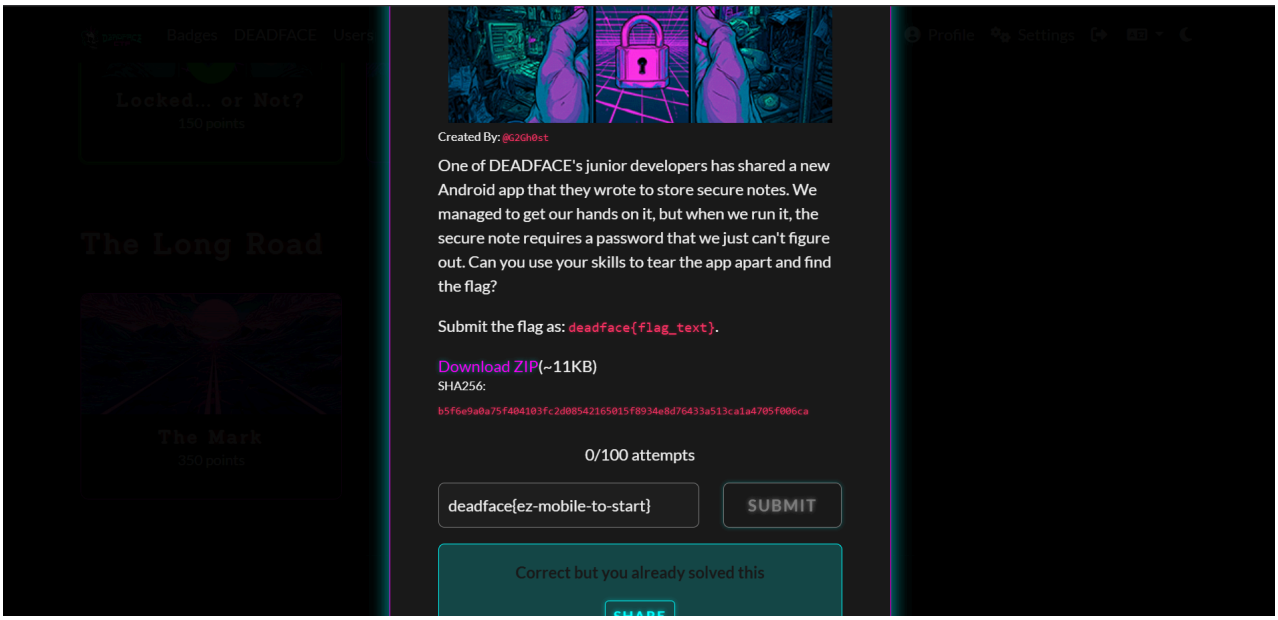
Step 3 —Identify the Base64-Encoded Flag



Step 4 —Decode the Flag



Step 5 —Verify the Result



5 Flag

deadface{ez-mobile-to-start}

6 Lessons Learned & Remediation

6.1 Key Takeaways

- **Static Analysis:** JADX-GUI can be used to decompile Android applications and inspect their source code for hardcoded strings, encoded data, and potentially exposed sensitive information.
- **Encoding vs. Encryption:** Base64 is an encoding scheme, not a security mechanism. Encoded data can be easily decoded without a secret key, making it unsuitable for protecting sensitive information.
- **Efficient Investigation:** Inspecting the Main Activity early in the analysis can help identify relevant application logic and potentially exposed data without requiring dynamic analysis.
- **Security Through Design:** Sensitive information, including credentials, cryptographic secrets and challenge flags, should not be embedded directly in client-side application code.

6.2 Remediation

1. **Avoid Hardcoding Sensitive Information:** Do not embed passwords, API keys, private keys, or other sensitive information directly in the application's source code or resources.
2. **Use Appropriate Cryptographic Protection:** Protect sensitive data using established cryptographic algorithms and secure key-management practices. Do not rely on Base64 encoding or obfuscation as a substitute for encryption.
3. **Implement Server-Side Security Controls:** Keep privileged operations and sensitive secrets on trusted server-side systems. Enforce authentication and authorization on the server rather than relying solely on client-side checks.
4. **Review Applications Before Release:** Integrate static analysis and secret-scanning tools into the development pipeline to identify hardcoded secrets and other security weaknesses before deployment.

7 References & Appendix

7.1 References

The following resources were used or are relevant to the tools and techniques discussed in this analysis.

1. JADX — Dex to Java Decompiler
   https://github.com/skylot/jadx
   Used to inspect the decompiled source code of the Android application.
2. Android Developers — App Manifest Overview
   https://developer.android.com/guide/topics/manifest/manifest-intro
   Reference for understanding the Android application manifest and its components.
3. RFC 4648 — The Base16, Base32, and Base64 Data Encodings
   https://datatracker.ietf.org/doc/html/rfc4648
   Technical reference for understanding Base64 encoding and decoding.

7.2 Appendix

Used Tools:

Tool	Purpose
unzip	Inspect the contents of the APK
JADX-GUI	Decompile and inspect Android application code
base64	Decode the identified Base64-encoded string
Kali Linux	Analysis environment

Evidence

The following evidence can be included to support the analysis:

- **Appendix A:** Terminal output showing the APK archive contents.
- **Appendix B:** JADX-GUI screenshot showing the relevant Main Activity code.
- **Appendix C:** The Base64-encoded string identified during analysis.
- **Appendix D:** Terminal output showing the decoded flag.

End of Writeup